

Module 4 – NON LINEAR DATA STRUCTURES

Trees

Binary Trees – Definition and its Properties

A binary tree is a hierarchical data structure in computer science where each node has at most two children, commonly referred to as the left child and the right child.

Key Components of a Binary Tree:

Node:

Each element in a binary tree is a node, which typically contains data and pointers (or references) to its left and right children.

Root:

The topmost node of the binary tree, serving as the starting point for operations and traversal.

Children:

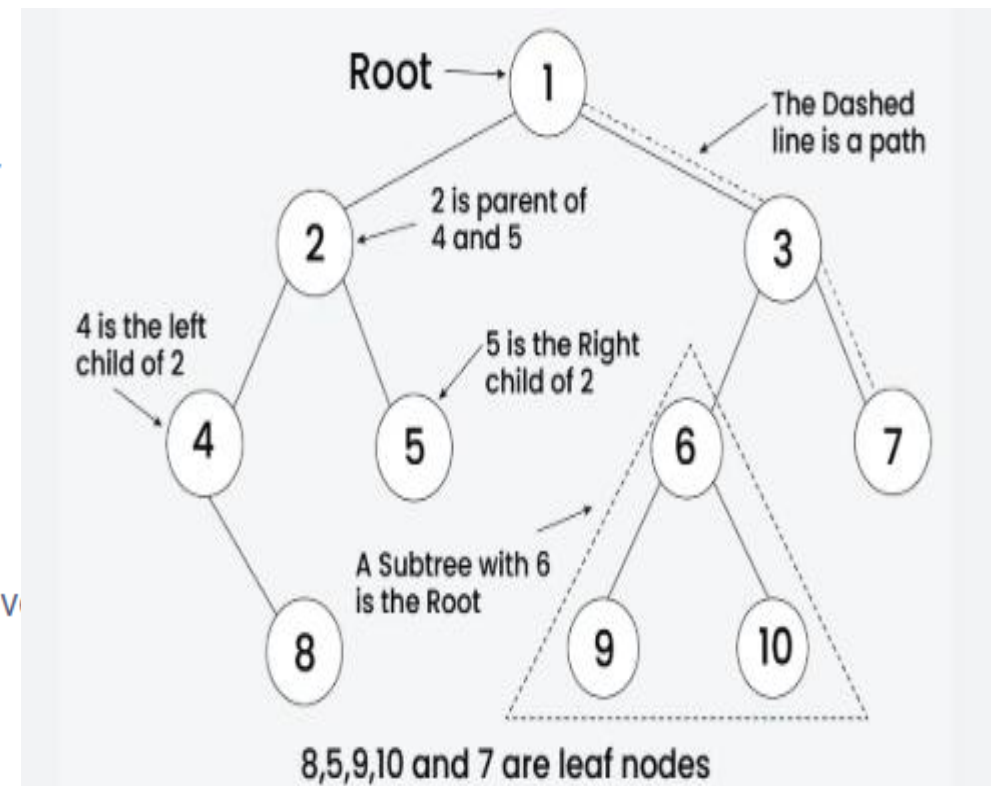
Nodes directly connected below a parent node. In a binary tree, a parent node can have zero, one, or two children. [🔗](#)

Leaf Node:

A node that has no children. These are the terminal nodes in the tree structure.

Subtree:

The left or right child of a node, along with all of its descendants, forms a subtree.

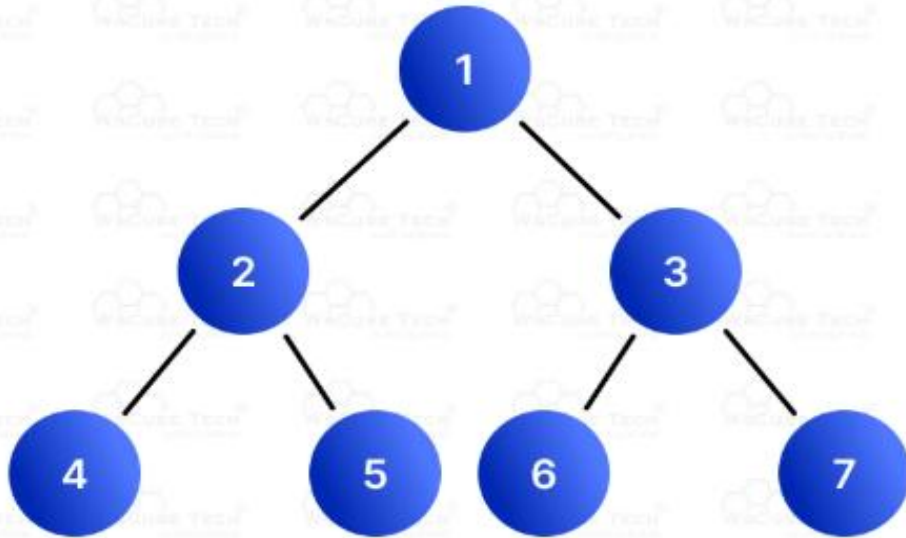


Types of Binary Tree in Data Structure

•Full Binary Tree

A full binary tree is a tree where every node has either 0 or 2 children.

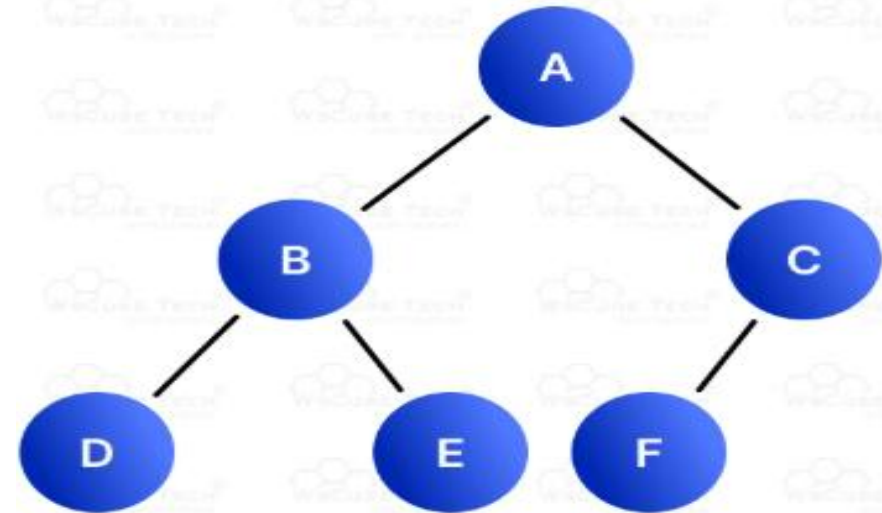
Full Binary Tree



•Complete Binary Tree

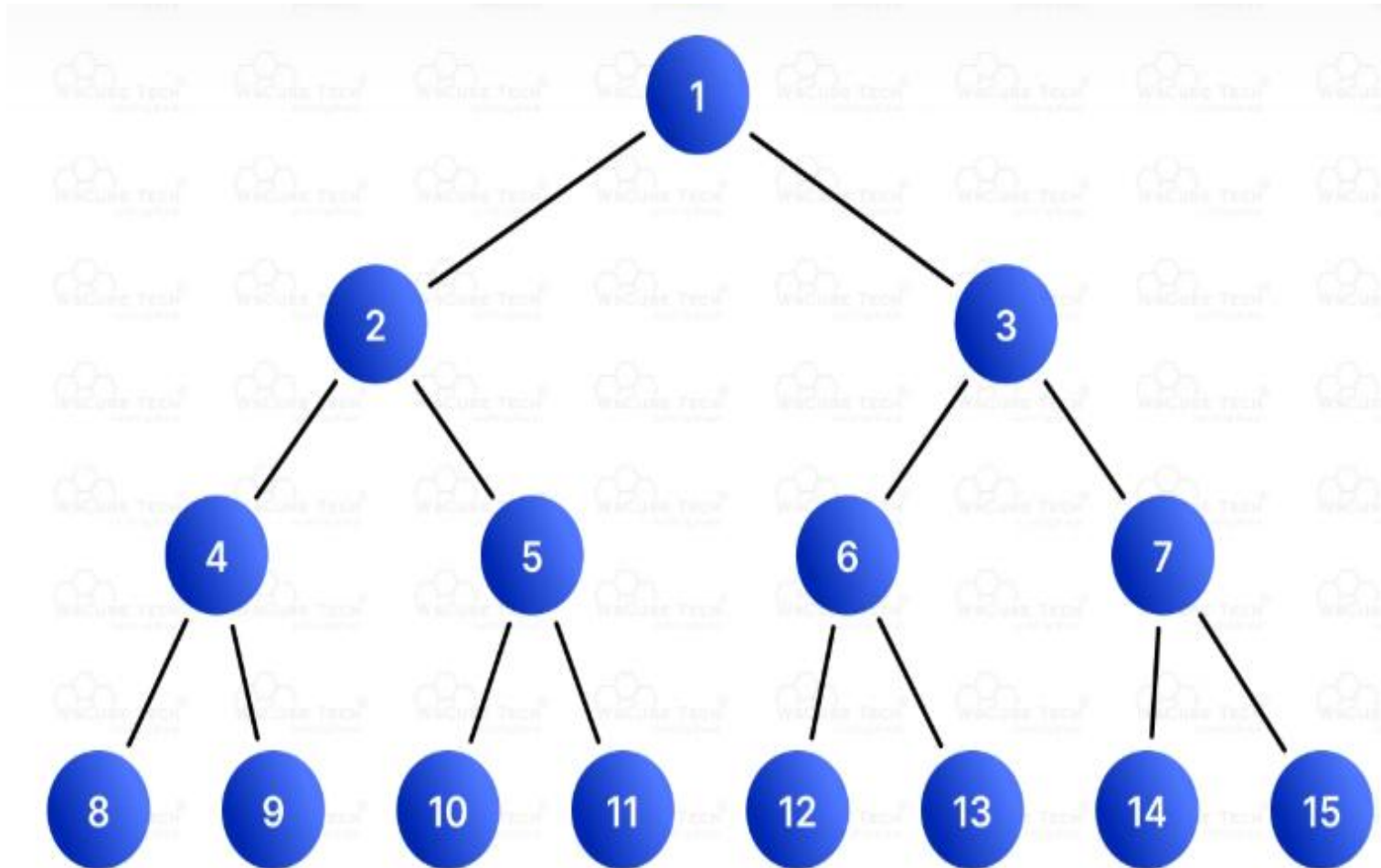
A complete binary tree is a tree where all levels are fully filled except possibly the last level, which is filled from left to right.

Complete Binary Tree

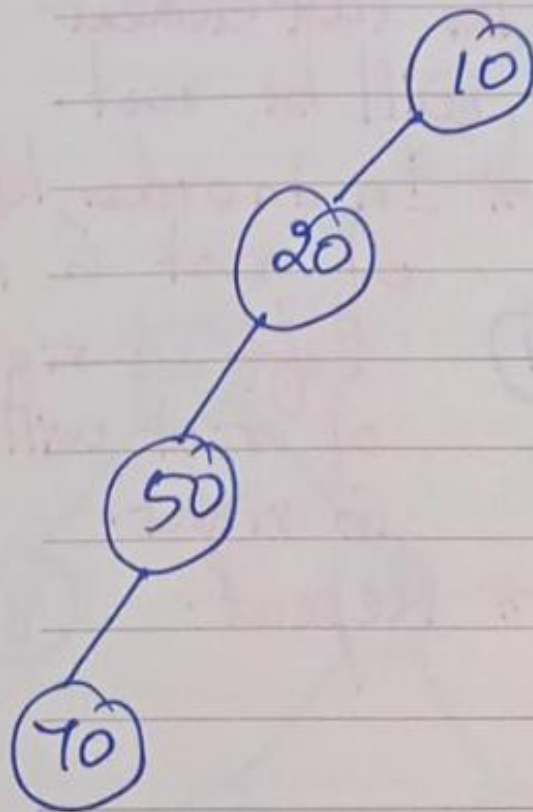


- **Perfect Binary Tree**

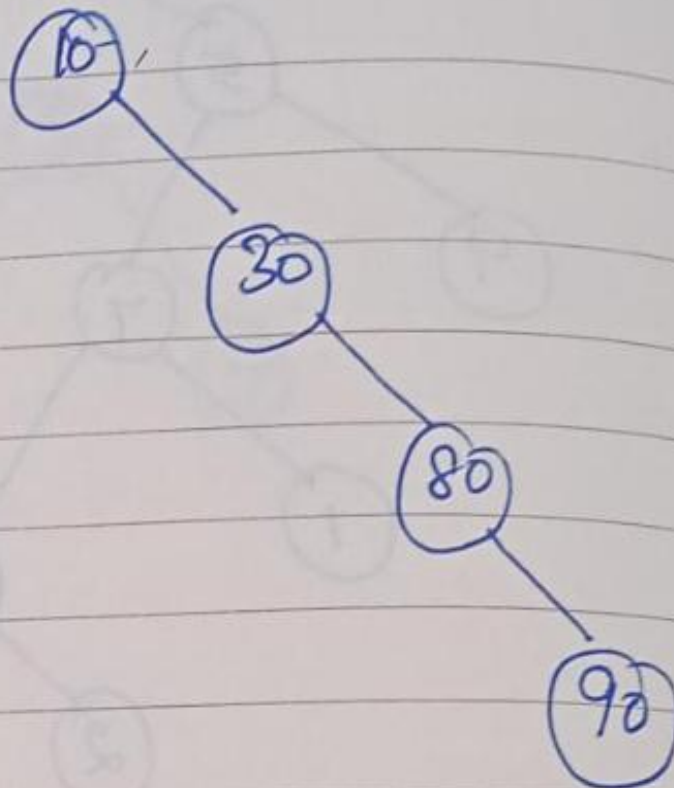
A perfect binary tree is a tree where **all internal nodes have exactly two children** and **all leaf nodes are at the same level**.



Left-skewed Tree



Right-skewed Tree



Depth of a Node:

- The depth of a specific node is the **number of edges on the path from the root node to that particular node**. The root node itself has a depth of 0.

Height of a Node:

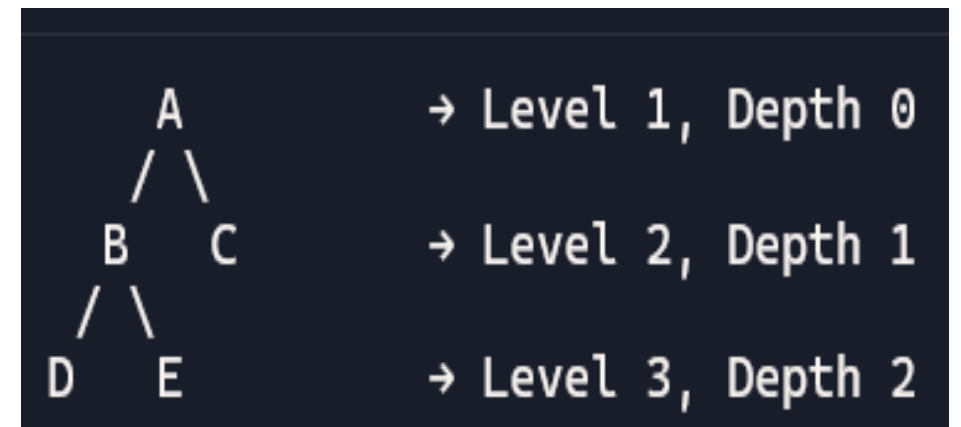
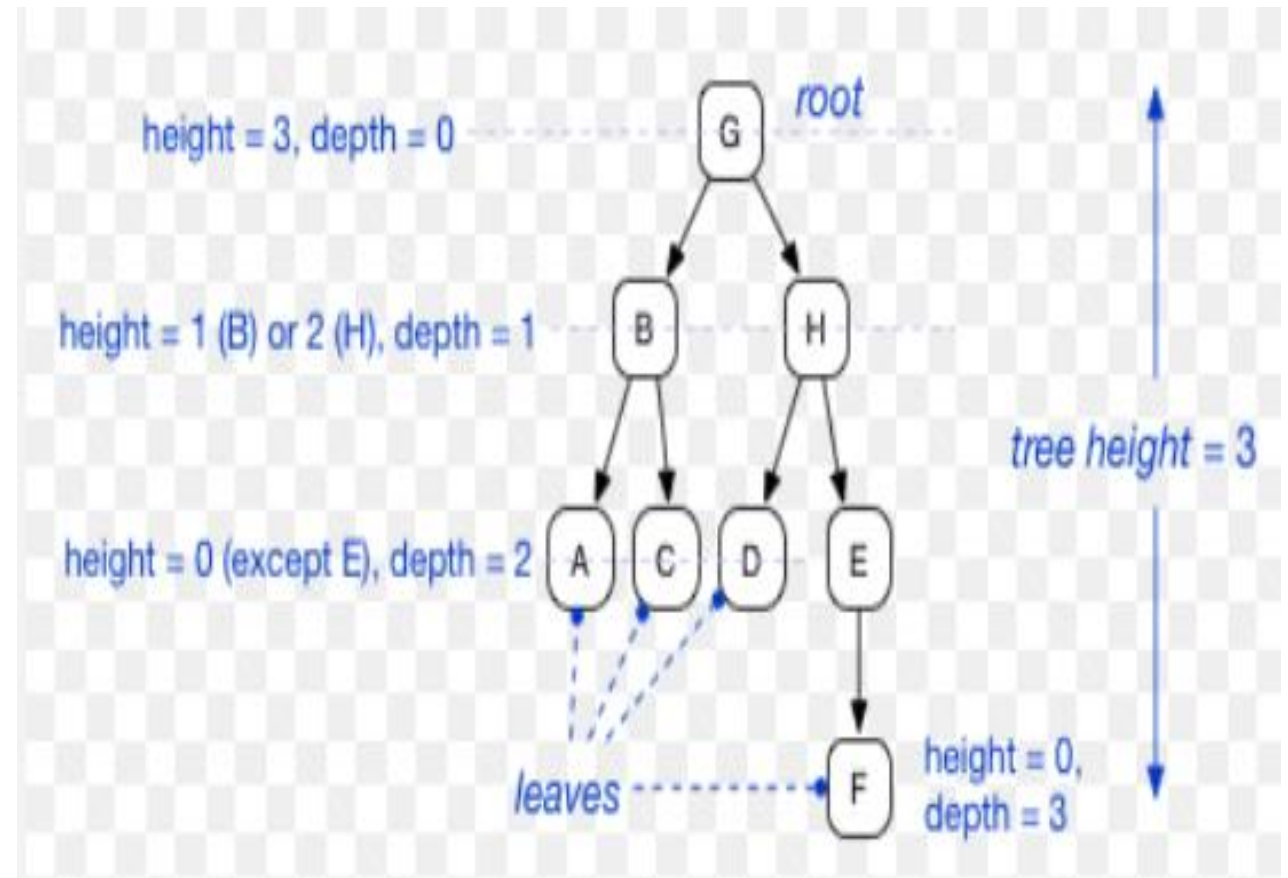
- The height of a specific node is the **number of edges on the longest path from that node to a leaf node in its subtree**. A leaf node (a node with no children) has a height of 0.

Height of a Tree:

- The height of the entire binary tree is the height of its root node. Equivalently, it is the maximum depth of any node in the tree. It represents the **number of edges on the longest path from the root to any leaf node**.

Level of a node

- The **distance of a node from the root**. Starts counting from **1** (root is at Level 1)



Representation of Binary Trees

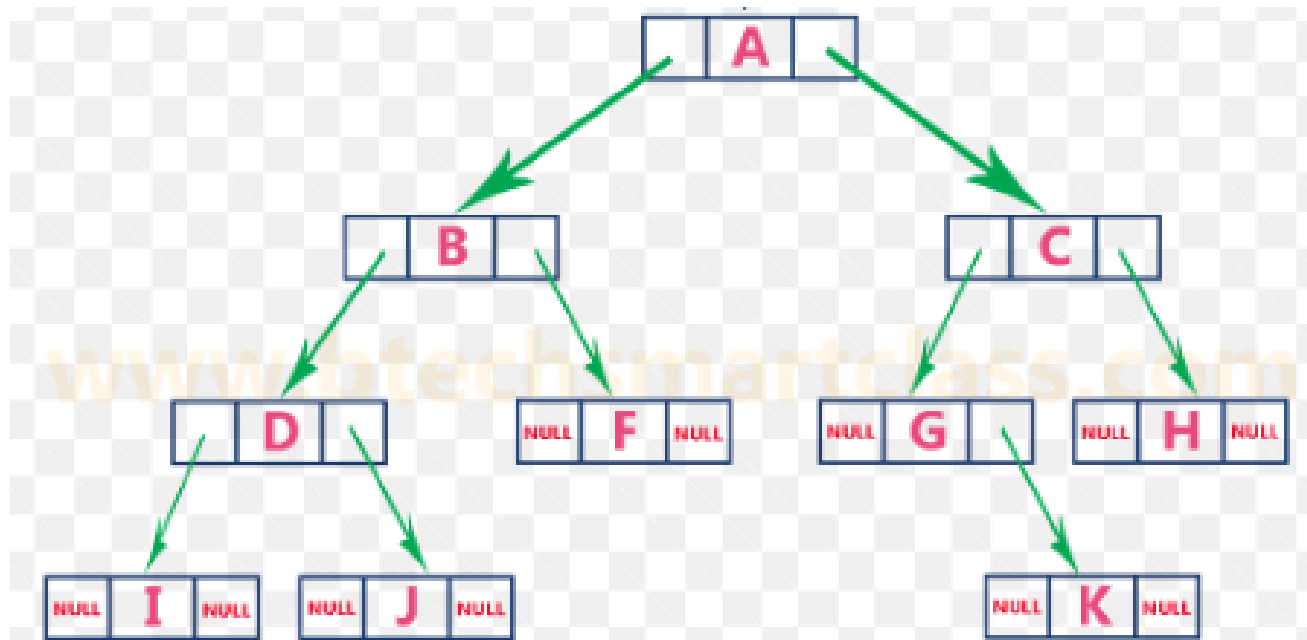
There are two primary ways to represent binary trees:

1. Linked Node Representation

2. Array Representation

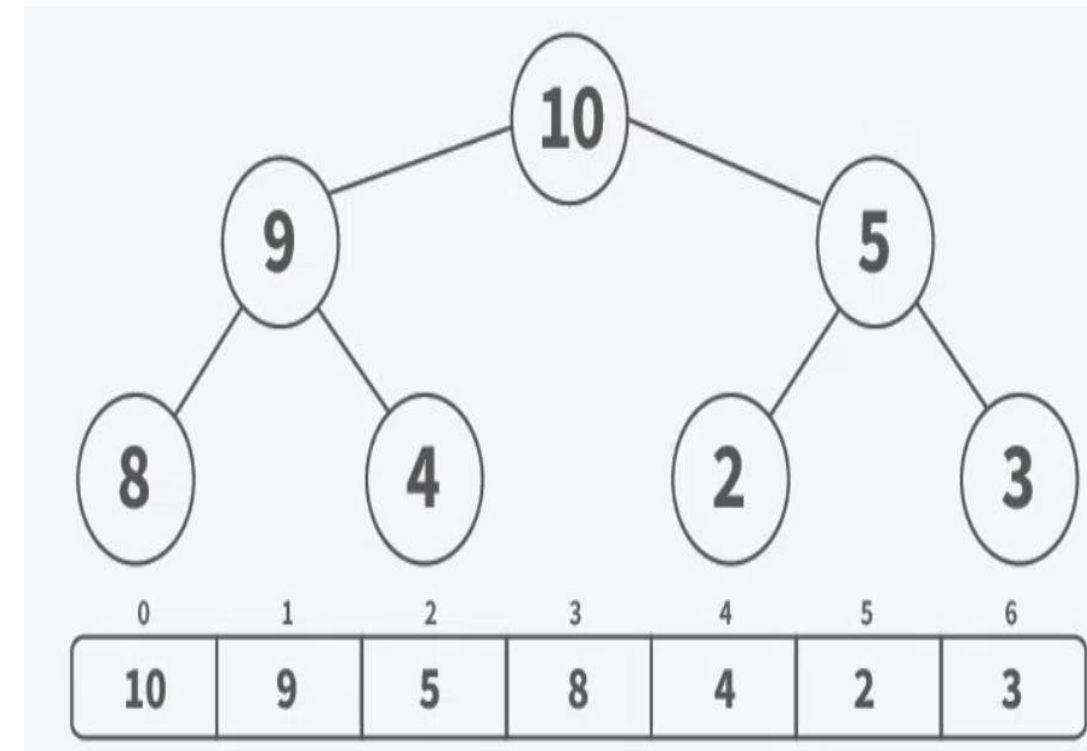
1. Linked Node Representation

This is the simplest way to represent a binary tree. Each node contains **data** and pointers to its **left** and **right** children.



2. Array Representation

- The tree is stored in an array.
- For any node at index i :
 - Left Child: Located at $2 * i + 1$
 - Right Child: Located at $2 * i + 2$
 - Its Parent is at: $\text{floor}((i-1) / 2)$
- Root Node: Stored at index 0



Maximum Nodes in a Binary Tree of Height 'h'

A binary tree of height **h** can have at most $2^{h+1} - 1$ nodes.

Example:

- For a binary tree of height $h = 0$ (only a root node), the maximum nodes are $2^{0+1} - 1 = 2^1 - 1 = 1$.
- For a binary tree of height $h = 1$, the maximum nodes are $2^{1+1} - 1 = 2^2 - 1 = 3$.
- For a binary tree of height $h = 2$, the maximum nodes are $2^{2+1} - 1 = 2^3 - 1 = 7$.

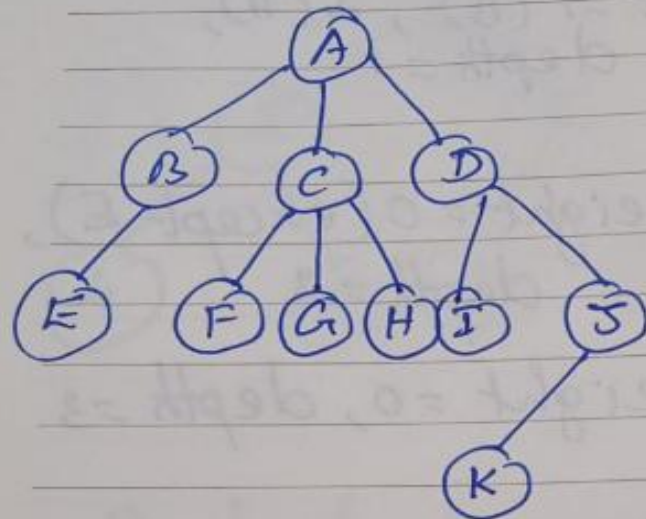
Generic Tree to Binary Tree

Convert the given tree to Binary Tree

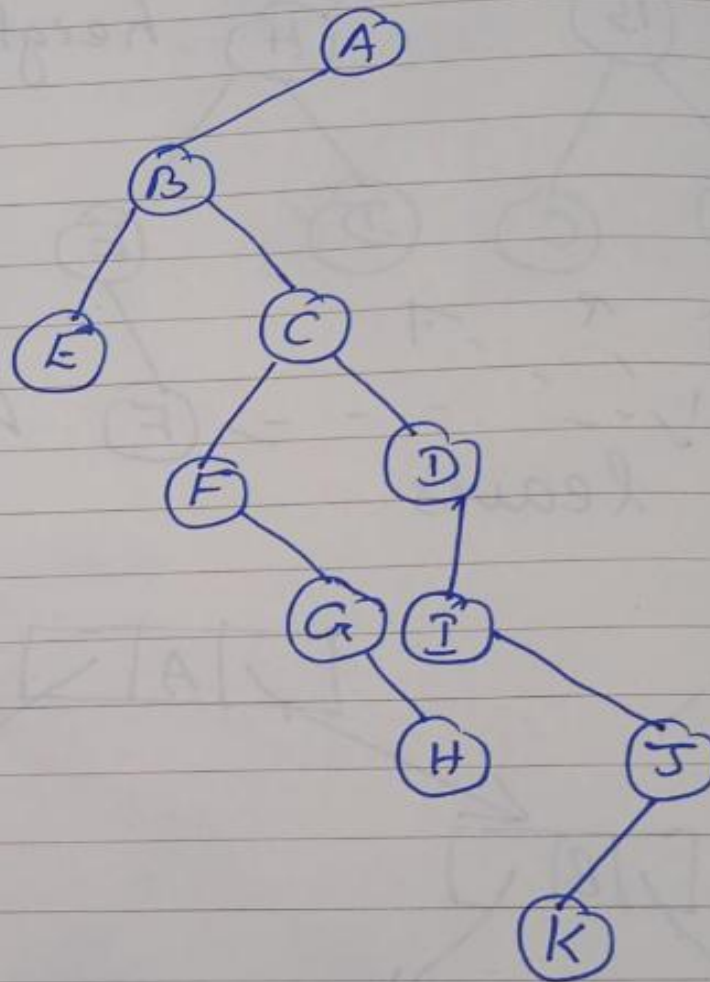
Steps:

- 1. Root of generic tree (n-ary tree) = Root of Binary Tree**
2. The left child of a node in the Generic Tree is the Left child of that node in the Binary Tree.
3. The right sibling of any node in the Generic Tree is the Right child of that node in the Binary Tree.

General Tree



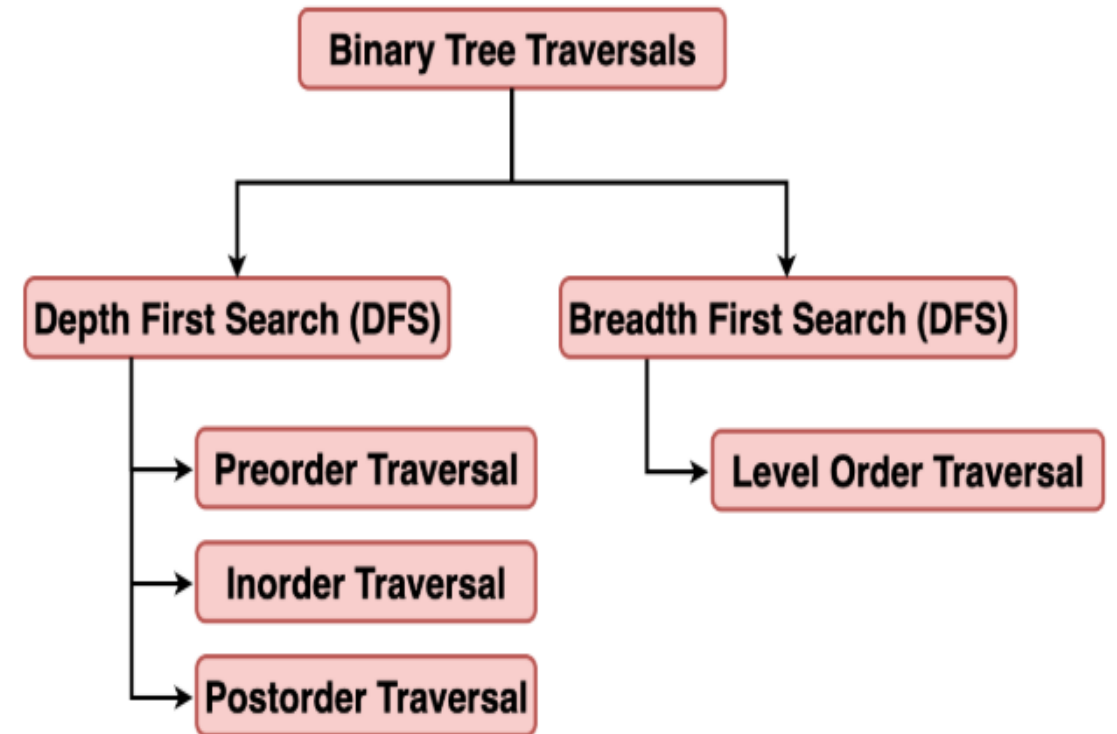
Binary Tree



Binary Tree Traversal

Traversing a binary tree means **visiting all the nodes in a specific order**. There are several traversal methods:

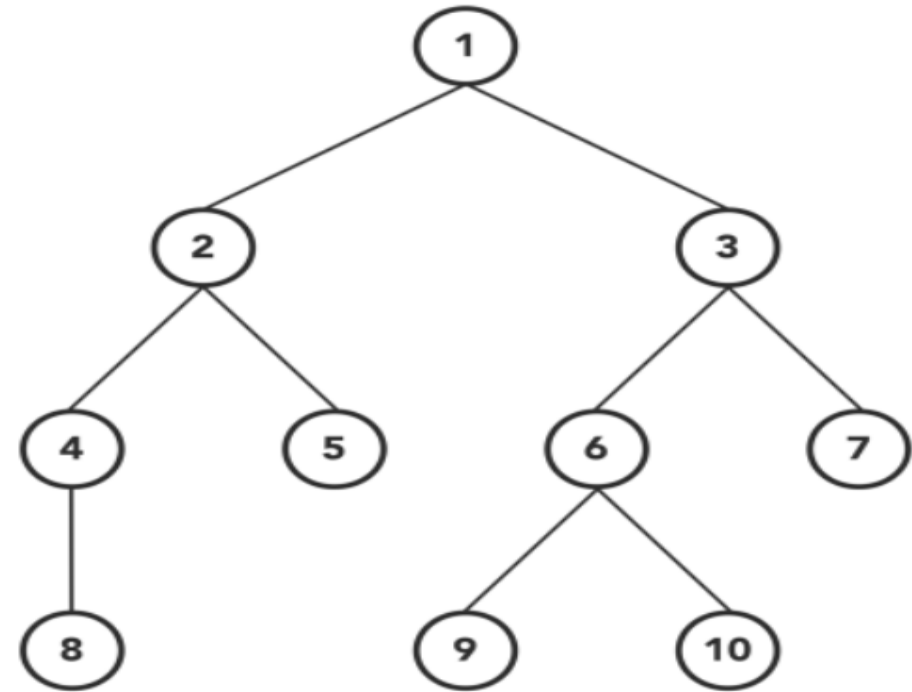
- in-order
- pre-order
- post-order
- level-order



1. In-Order Binary Tree Traversal

In in-order traversal, the left child is visited first, followed by the node itself, and then the right child. This can be visualized as **Left - Root - Right**.

```
void inOrderTraversal(struct Node* root) {  
    if (root == NULL) return;  
  
    inOrderTraversal(root->left);  
    printf("%d ", root->data);  
    inOrderTraversal(root->right);  
}
```



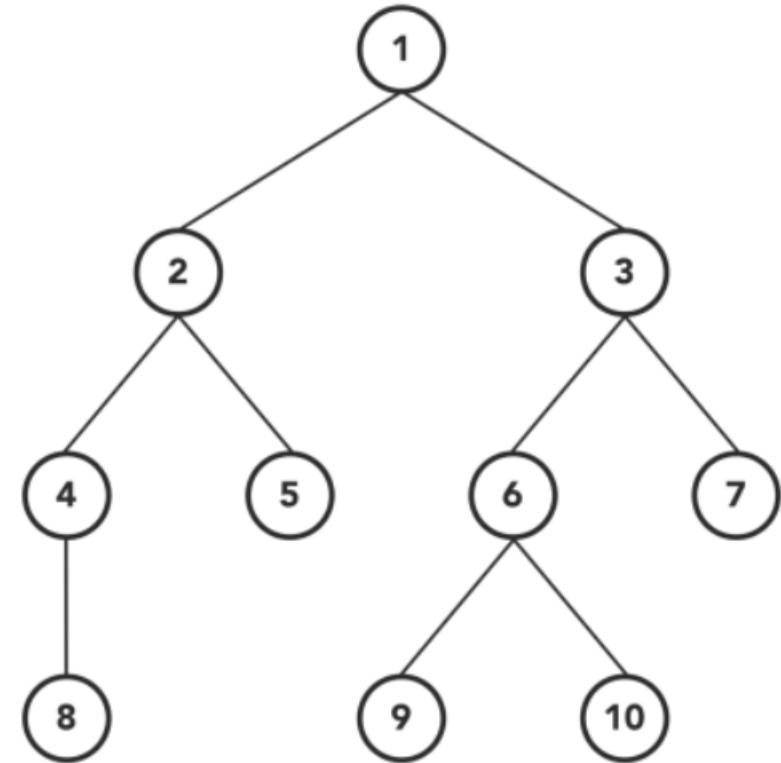
In order Traversal: 8 4 2 5 1 9 6 10 3 7

- In-order traversal (**Left → Root → Right**)
- Pre-order traversal (**Root → Left → Right**)
- Post-order traversal (**Left → Right → Root**)
- Level-order traversal (**Breadth First**)

2. Pre-Order Binary Tree Traversal

In pre-order traversal, the node is visited first, followed by its left child and then its right child. This can be visualized as **Root - Left - Right**.

```
void preOrderTraversal(struct Node* root) {  
    if (root == NULL) return;  
  
    // Visit the root node  
    printf("%d ", root->data);  
  
    // Traverse the left subtree  
    preOrderTraversal(root->left);  
  
    // Traverse the right subtree  
    preOrderTraversal(root->right);  
}
```

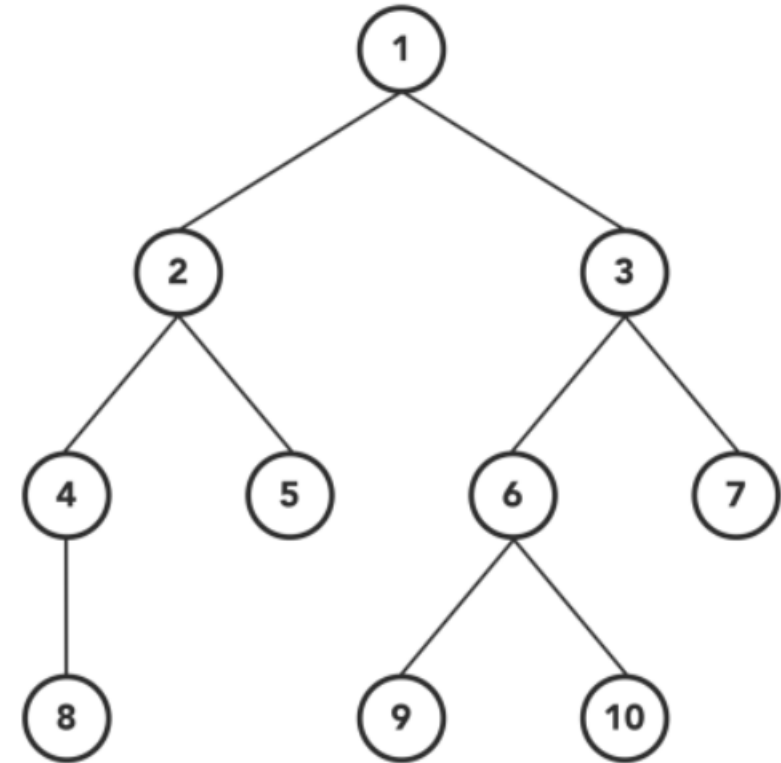


Pre order Traversal: 1 2 4 8 5 3 6 9 10 7

3. Post-Order Binary Tree Traversal

In post-order traversal, the left child is visited first, then the right child, and finally the node itself. This can be visualized as **Left - Right - Root**.

```
void postOrderTraversal(struct Node* root) {  
    if (root == NULL) return;  
  
    // Traverse the left subtree  
    postOrderTraversal(root->left);  
  
    // Traverse the right subtree  
    postOrderTraversal(root->right);  
  
    // Visit the root node  
    printf("%d ", root->data);  
}
```



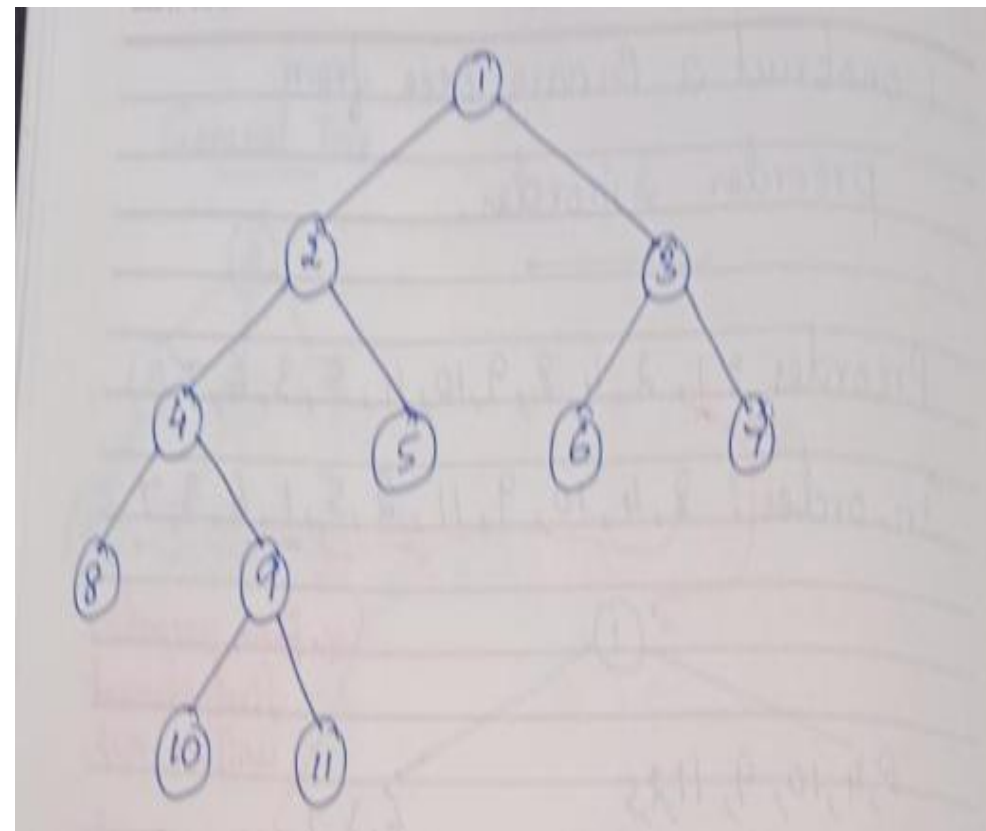
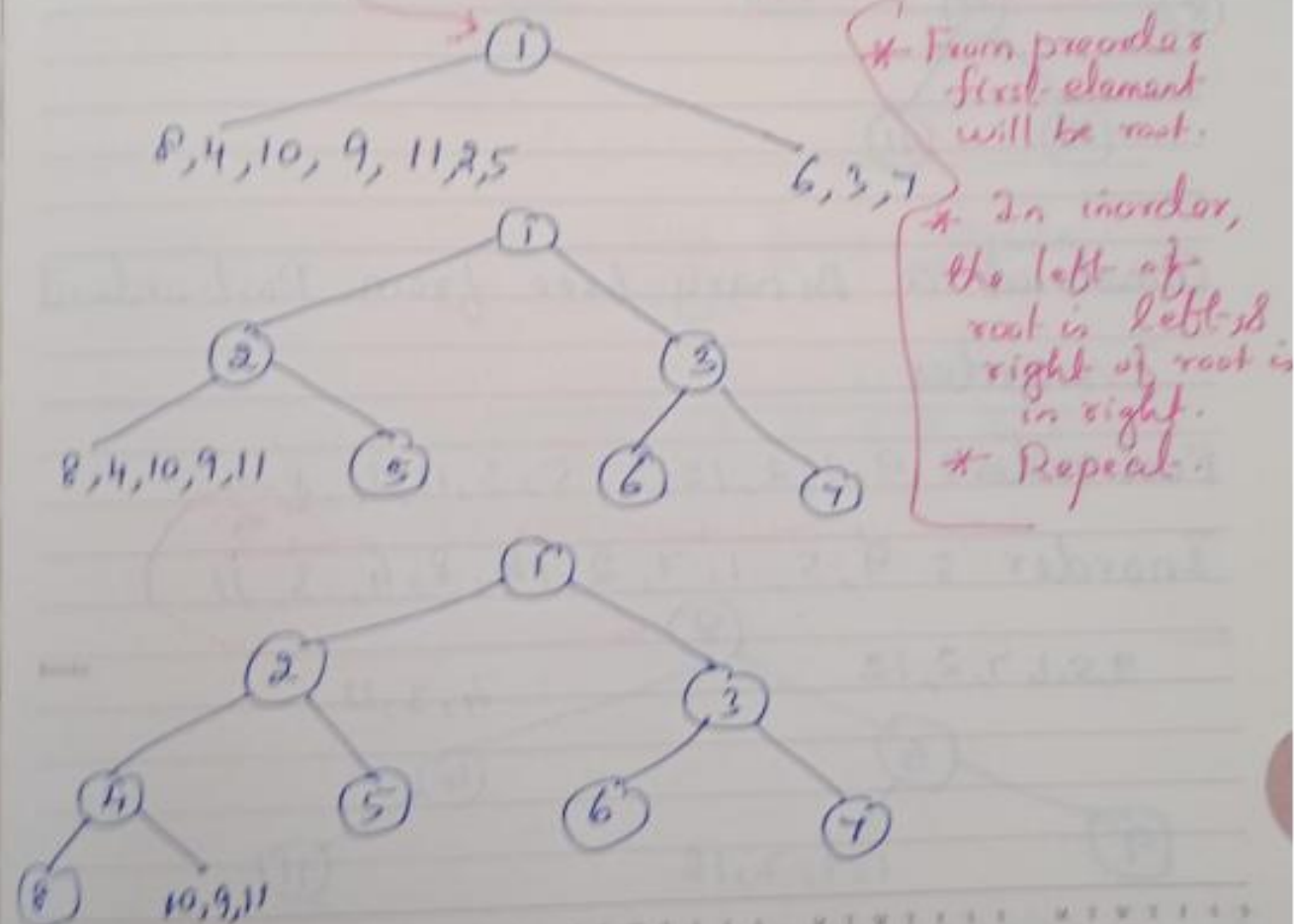
Pre order Traversal: 8 4 5 2 9 10 6 7 3 1

Level Order: 1 2 3 4 5 6 7 8 9 10

Construct a Binary tree from
preorder & inorder.

Preorder: 1, 2, 4, 8, 9, 10, 11, 5, 3, 6, 7

In-order: 8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7

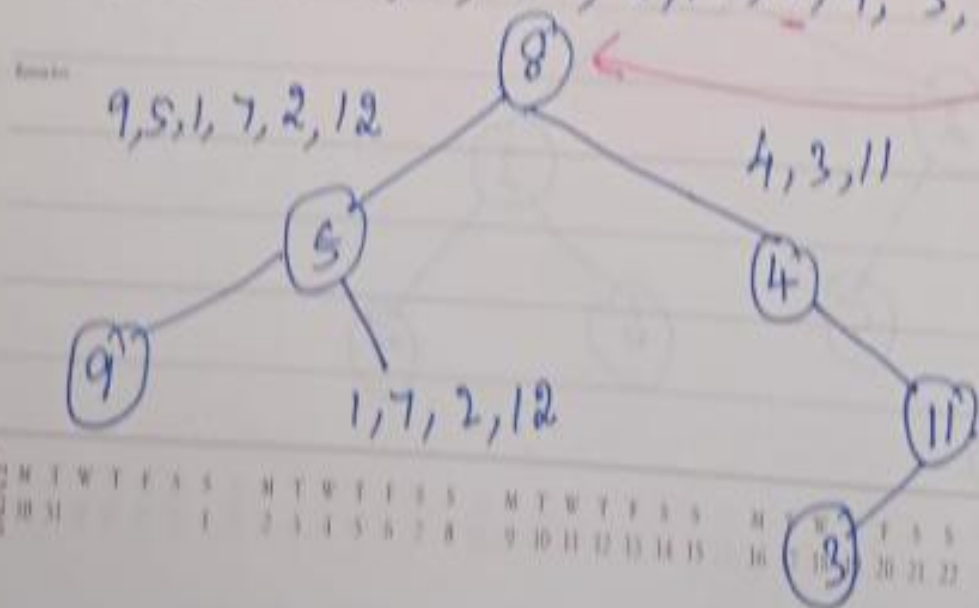


Construct a Binary tree from Post-order & In order:

In order:

Post order: 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

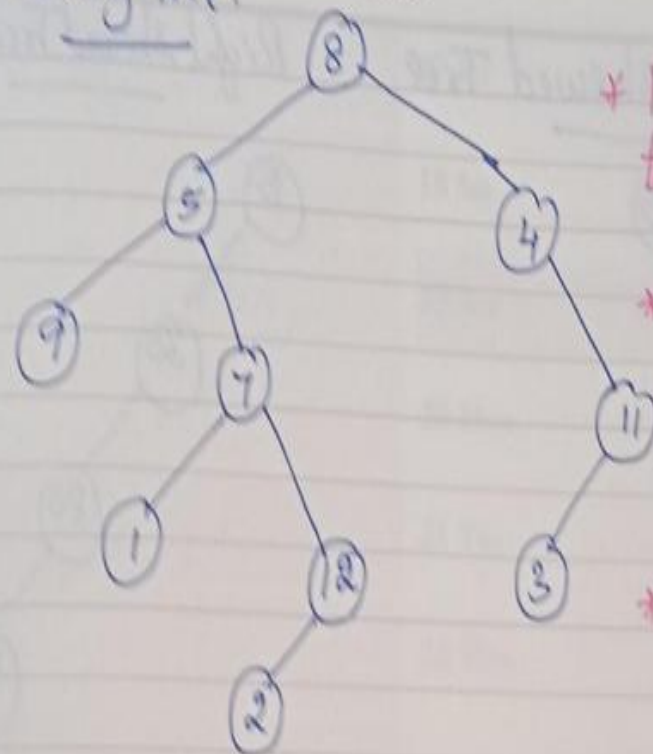
In order: 9, 5, 1, 7, 2, 12, 8, 4, 3, 11



Binary Tree:

30 Monday
MAY

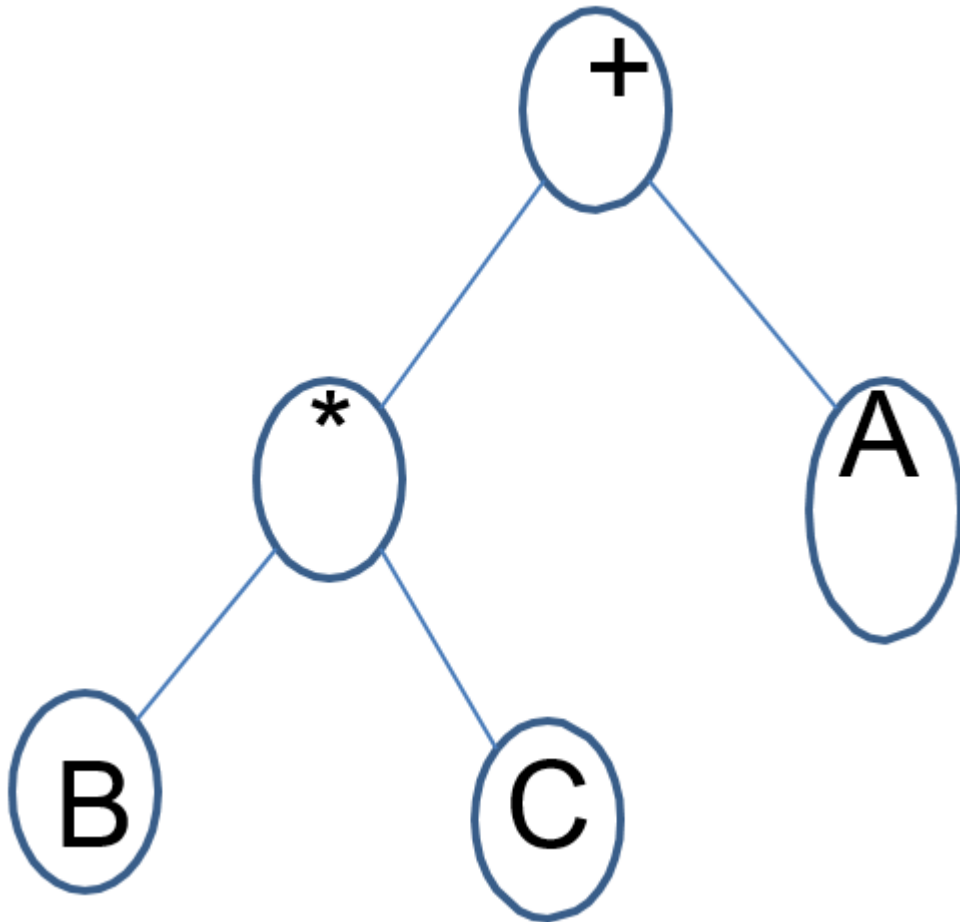
150-215 Week 22



* From postorder, the last element will be root.
* In Inorder, left of root is in left & right of root will be in right.
* Repeat.

Expression tree

An expression tree is a binary tree in which the operands are attached as leaf nodes and operators become the internal nodes.



Inorder traversal: $B * C + A$

Preorder traversal: $+ * B C A$

Postorder traversal: $B C * A +$

Creation of an expression tree

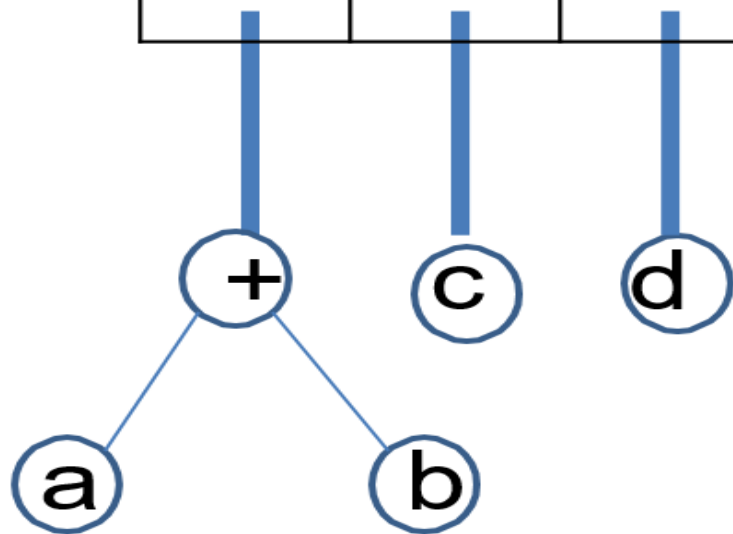
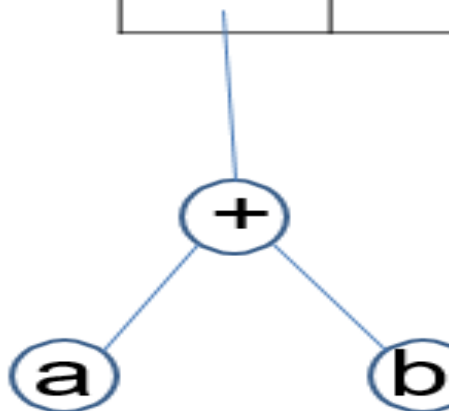
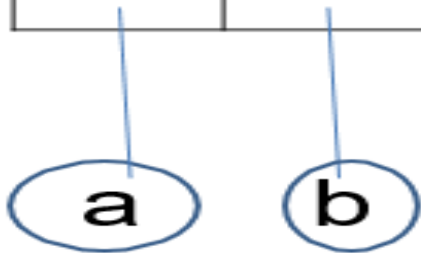
Convert the given **Postfix expression to expression tree**

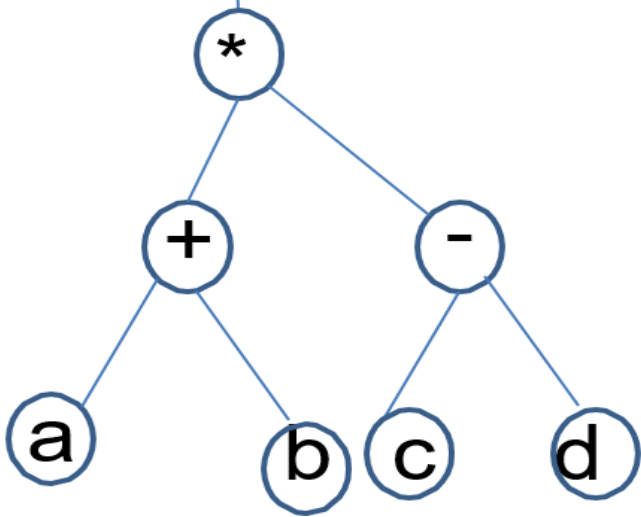
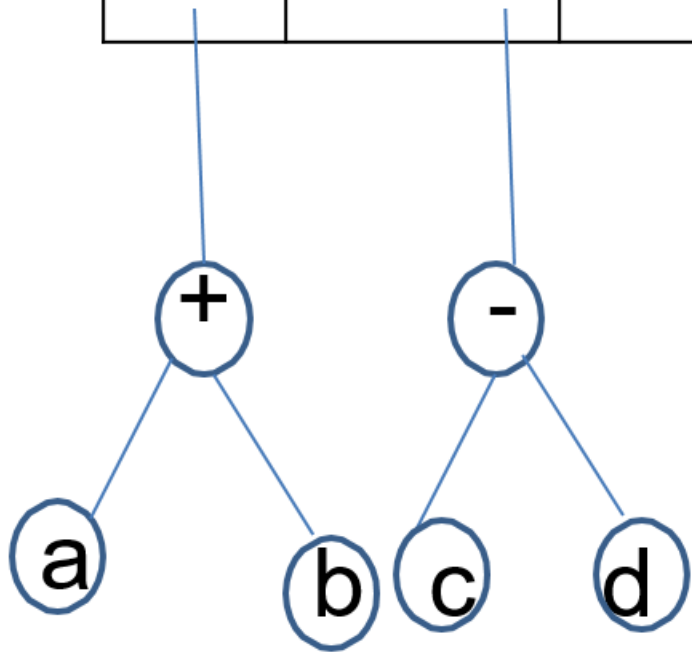
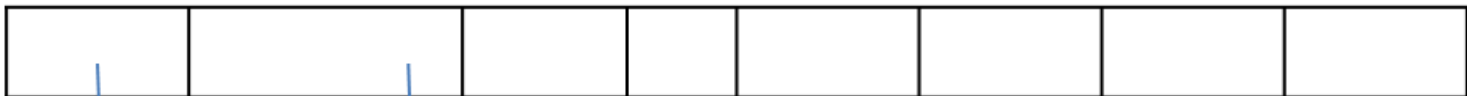
Postfix expression is: a b + c d - *

Step 1: Read each symbol from left to right one character at a time

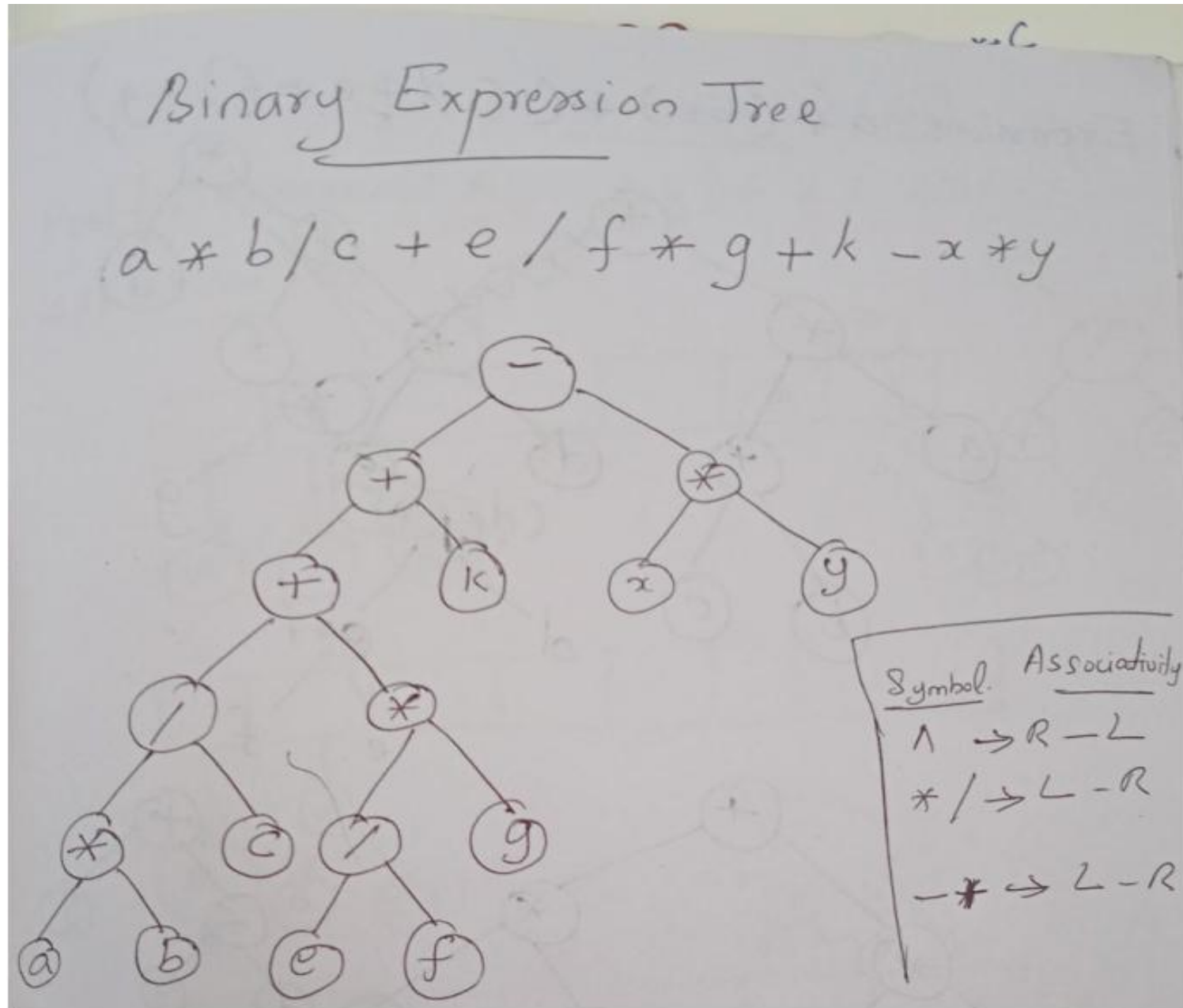
Step 2: if we read an operand then we will make a node of it and push it onto the stack.

Step 3: if we read operator then pop two nodes from stack, the first popped node will be attached as right child to operator node and second popped node will be attached as a left child to operator node





Convert the given **Infix expression to expression tree**

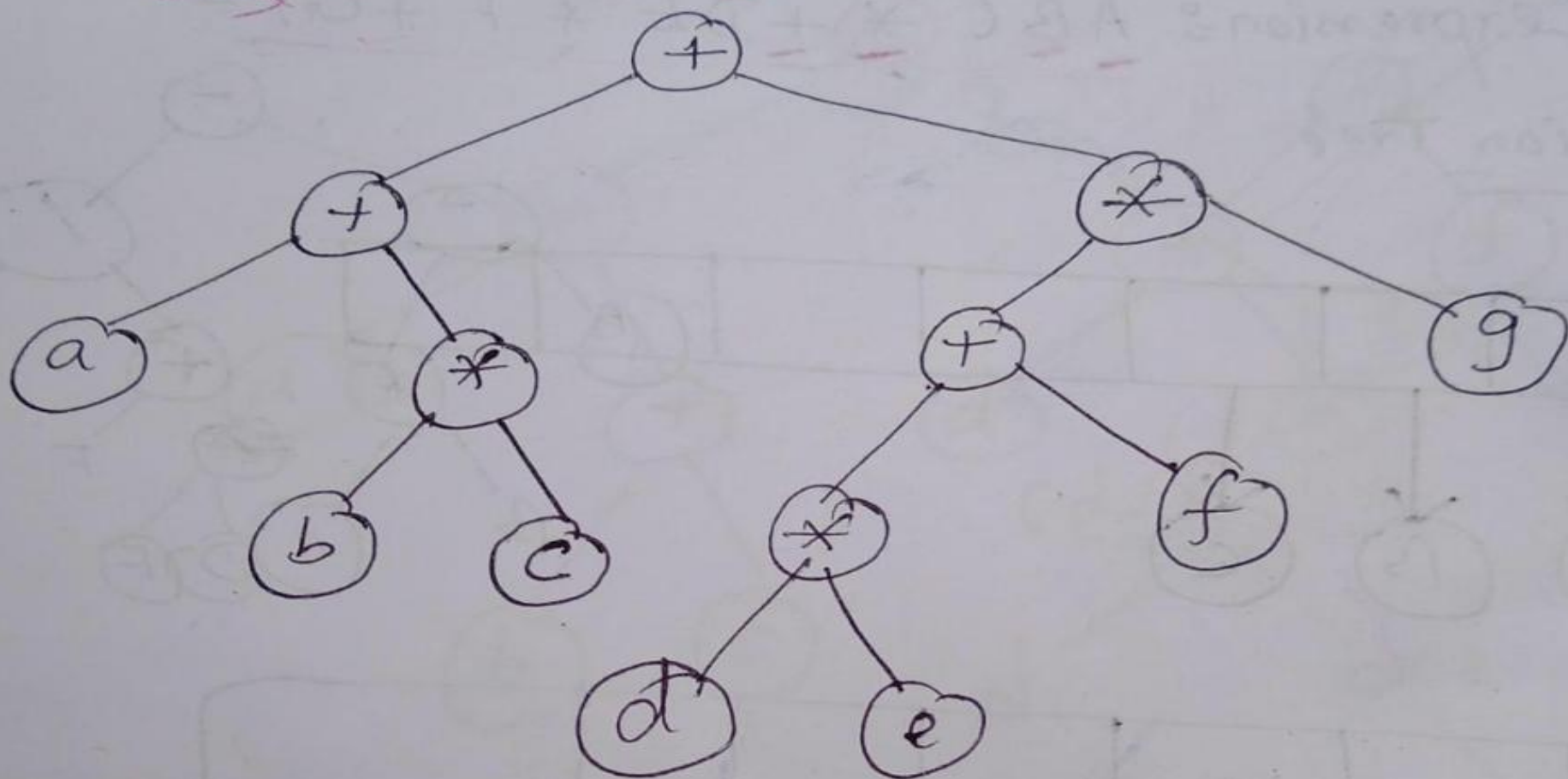


Step 1: Lowest precedence operator will be the root. (If more, Check Associativity)

Step 2: Left part of the expression will make left subtree and the right part will make right subtree.

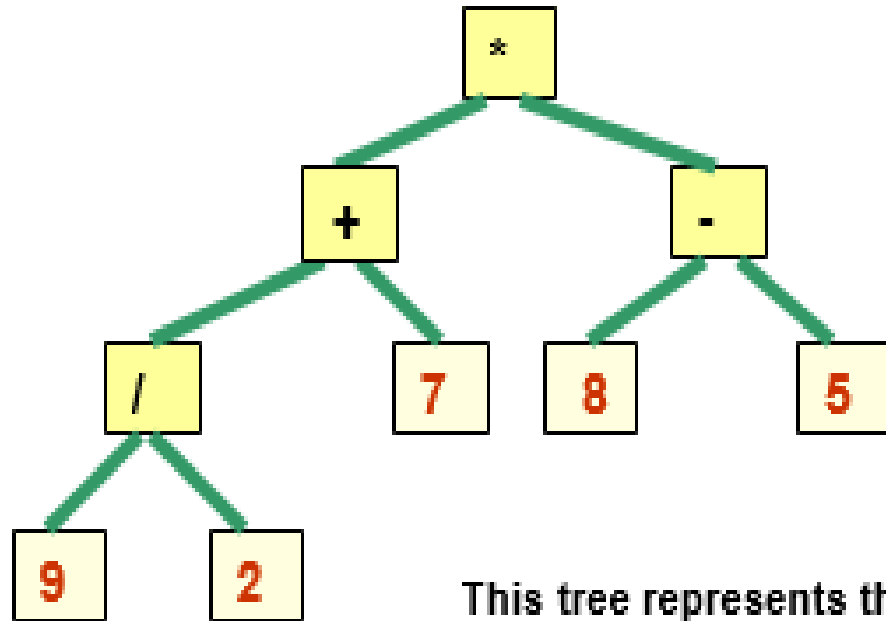
Step 3: Repeat from Step 1.

$$(a + b * c) + ((d * e + f) * g)$$



Construct an expression tree

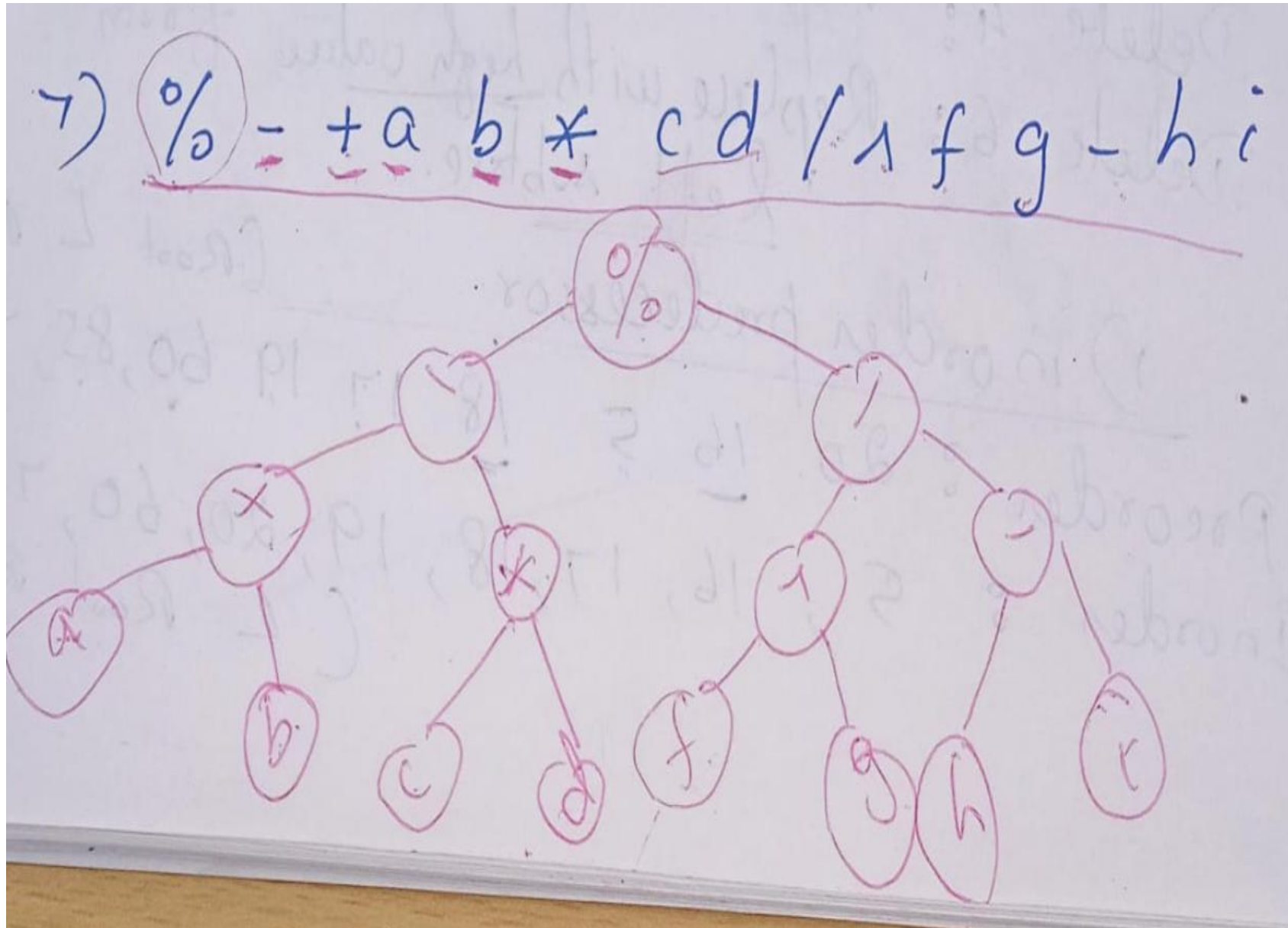
$(9 / 2 + 7) * (8 - 5)$



This tree represents the expression

$(9 / 2 + 7) * (8 - 5)$

Convert the given Prefix expression to expression tree



Step 1: First operator will be the root.

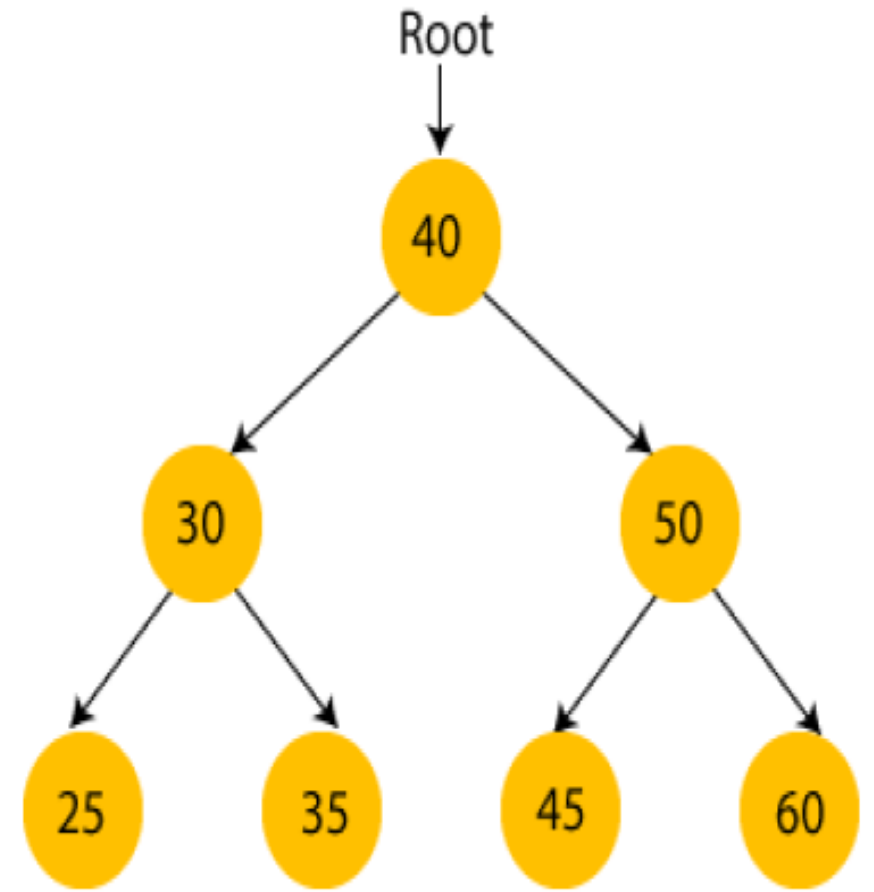
Step 2: Next subsequent operators are left until encounter an operand as leaf.

Step 3: Once leaf is encountered, move to right and repeat.

BINARY SEARCH TREE

Binary search tree is a binary tree in which the nodes are arranged in specific order.

That means the values at left subtree < root node value < values at right subtree.



Binary Search Tree – Operations

1.Insert

2.Delete

3.Find Min

4.Find Max

5.Kth Smallest Element

```
struct Node {  
    int key;  
    struct Node* left;  
    struct Node* right;  
};
```

```
struct Node* newNode(int item)  
{  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node));  
    temp->key = item;  
    temp->left = temp->right = NULL;  
    return temp;  
}
```

```
struct Node* insert(struct Node* node, int  
key) {
```

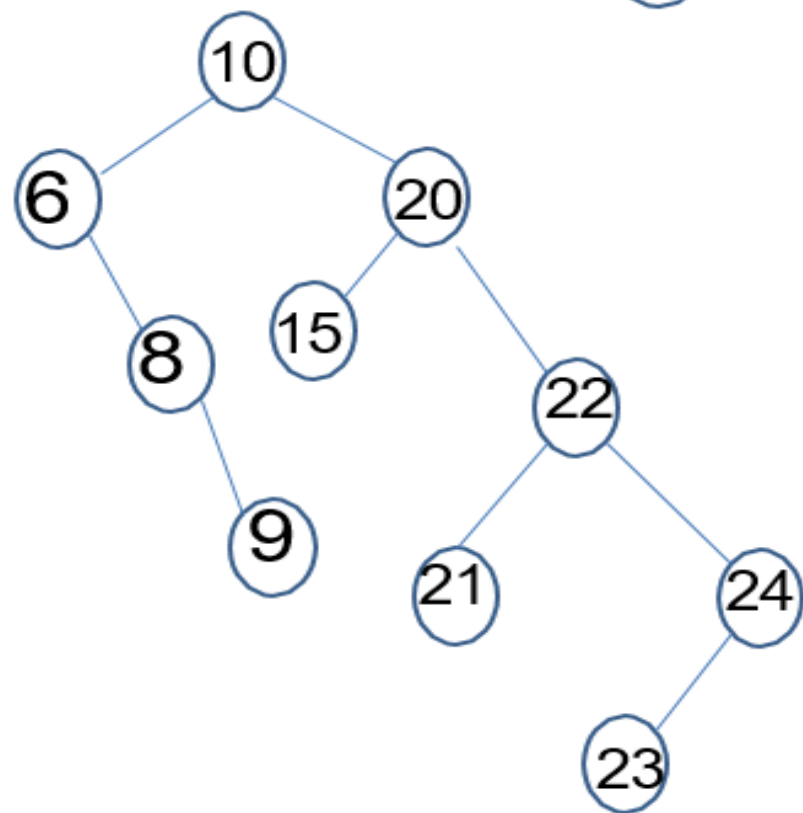
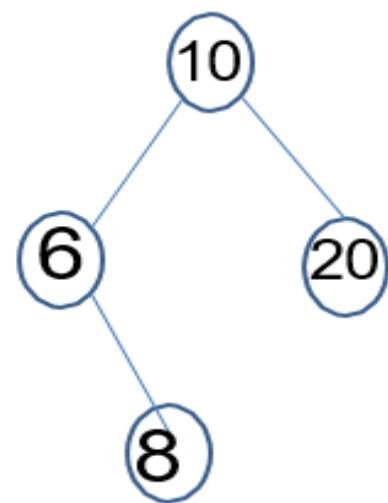
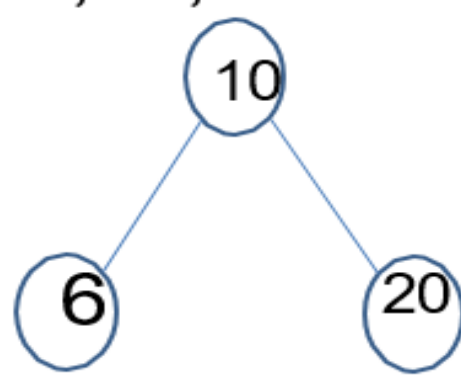
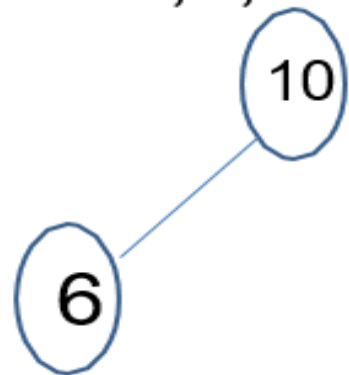
```
    // If the tree is empty, return a new node  
    if (node == NULL)  
        return newNode(key);  
    if (node->key < key)  
        node->right = insert(node->right, key);  
    else  
        node->left = insert(node->left, key);
```

```
    return node;  
}
```

```
int main() {  
    struct Node* root = newNode(50);  
    root = insert(root, 30);  
    root = insert(root, 20);  
    root = insert(root, 40);
```

```
    // Print inorder traversal of the BST  
    inorder(root);  
    return 0;  
}
```

10,6,20,8,9,15,22,21,24,23



Deletion of an element

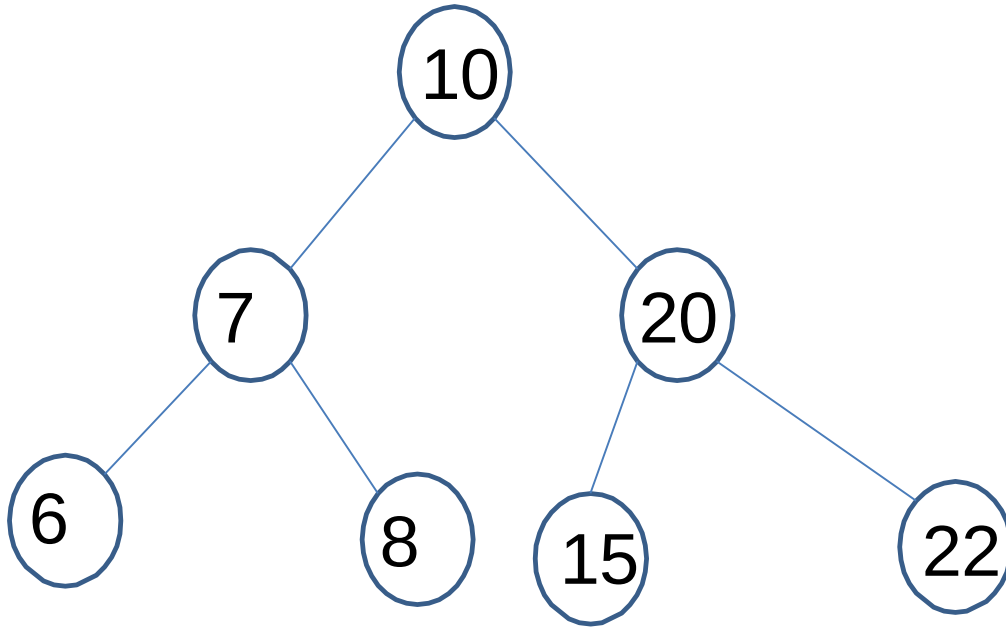
1. Deletion of leafnode
2. Deletion of node having one child
3. Deletion of a node having two children

1. Deletion of leafnode

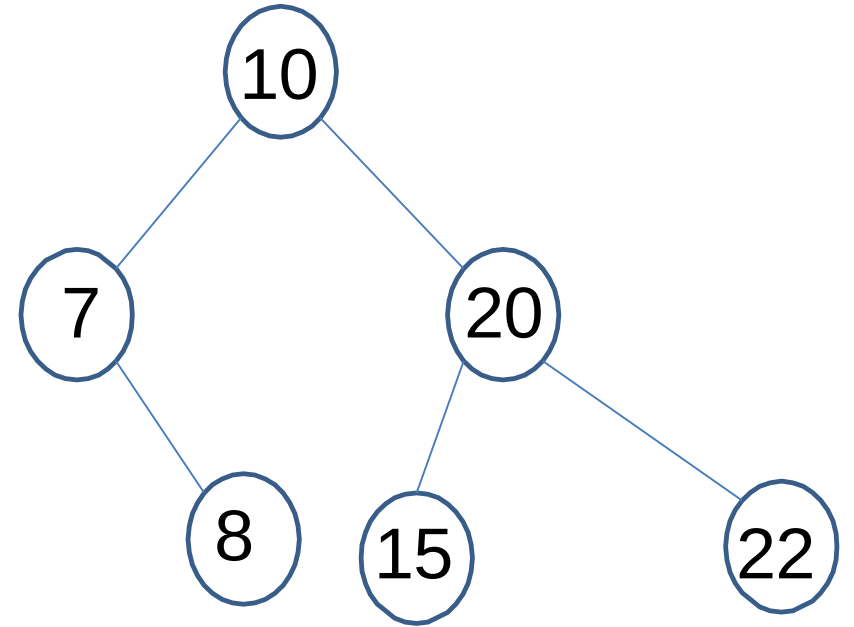
Delete the leafnode

Set the left or right pointer of parent to null

1. Deletion of leaf node 6



Before Deletion

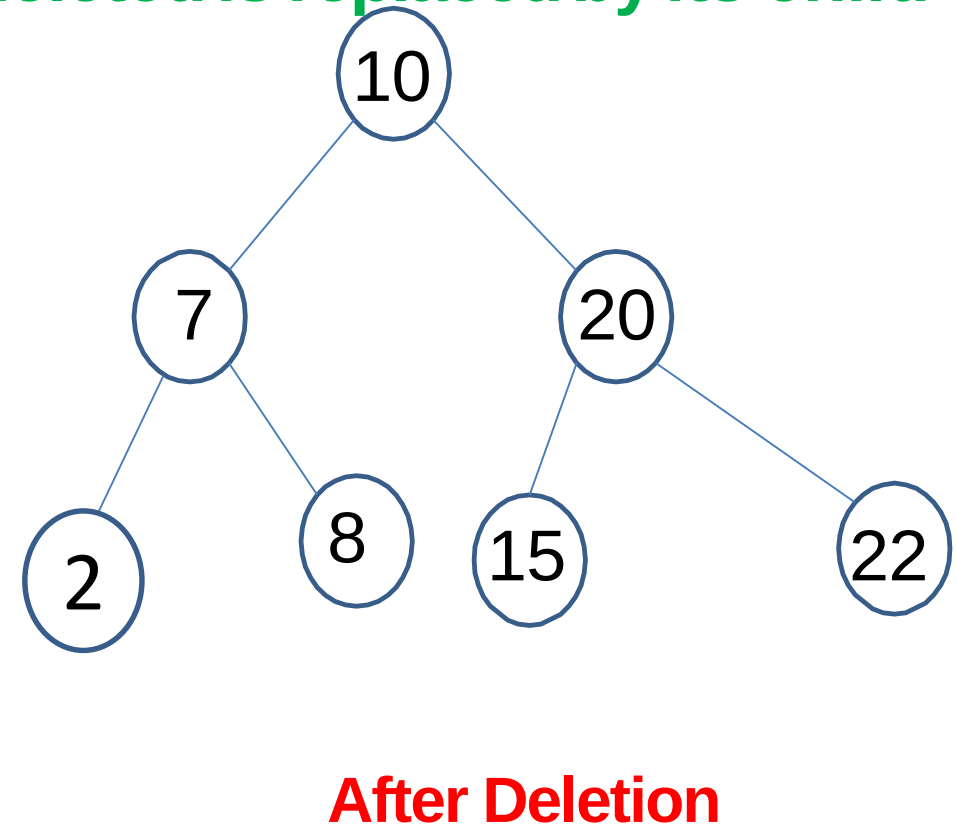
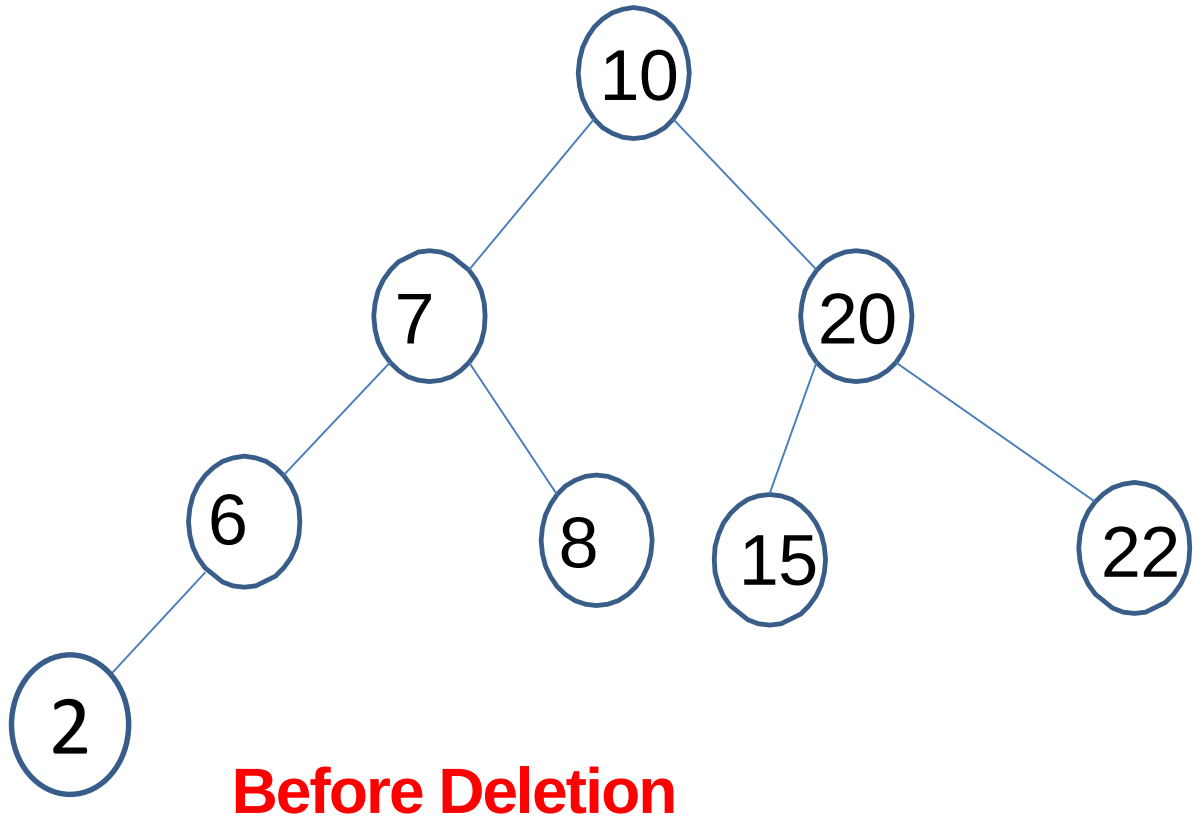


After Deletion

Deletion of node having one child

child is set as the child of the node's parent

Node to be deleted is replaced by its child



Deletion of node having two children

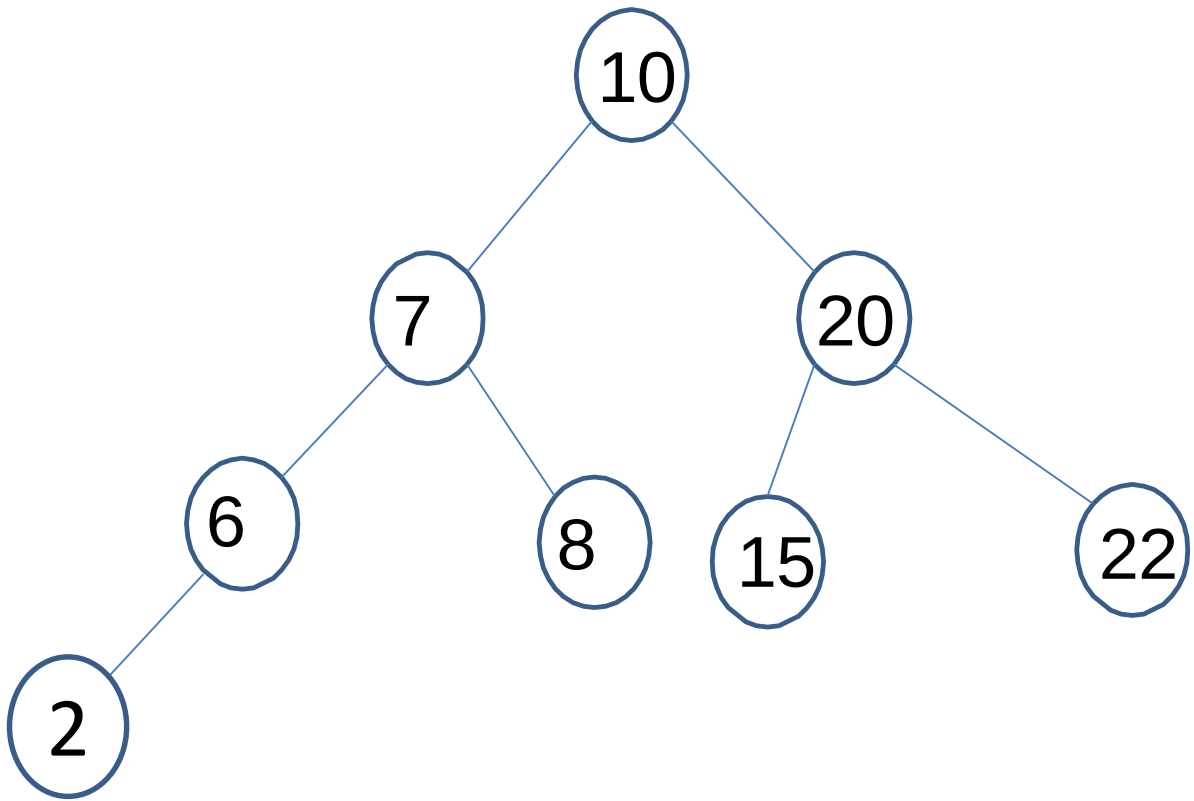
Find the inorder successor (Or inorder predecessor) of the node to be deleted

Copy it at the position of the node to be deleted

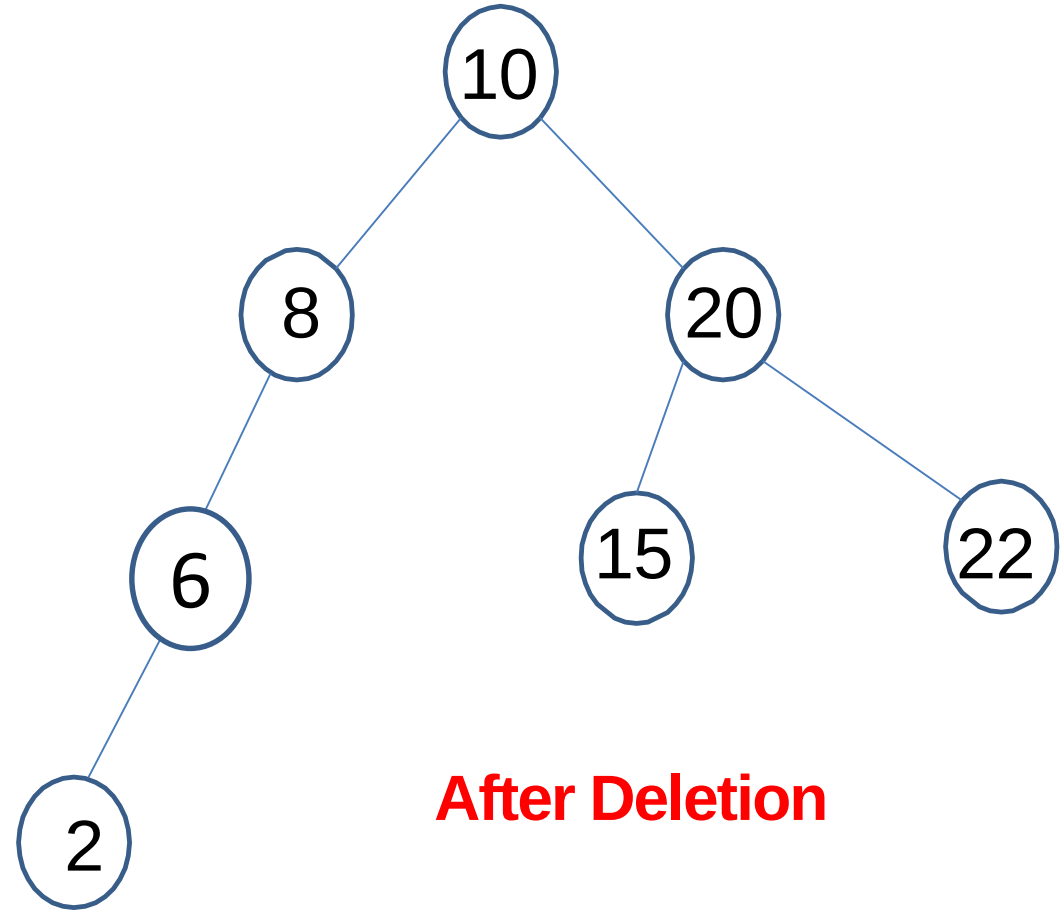
Deletion of node having two children: **delete 7**

3. Deletion of node 7

In Order is: 2, 6, 7, 8, 10, 15, 20, 22



Before Deletion



After Deletion

```

struct Node* deleteNode(struct Node* root,
int key) {
    if (root == NULL)
    {
        return root;
    }

    // Traverse to find the node to be deleted
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    else { // Node with key found
        // Case 1: No child or one child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
    }
}

```

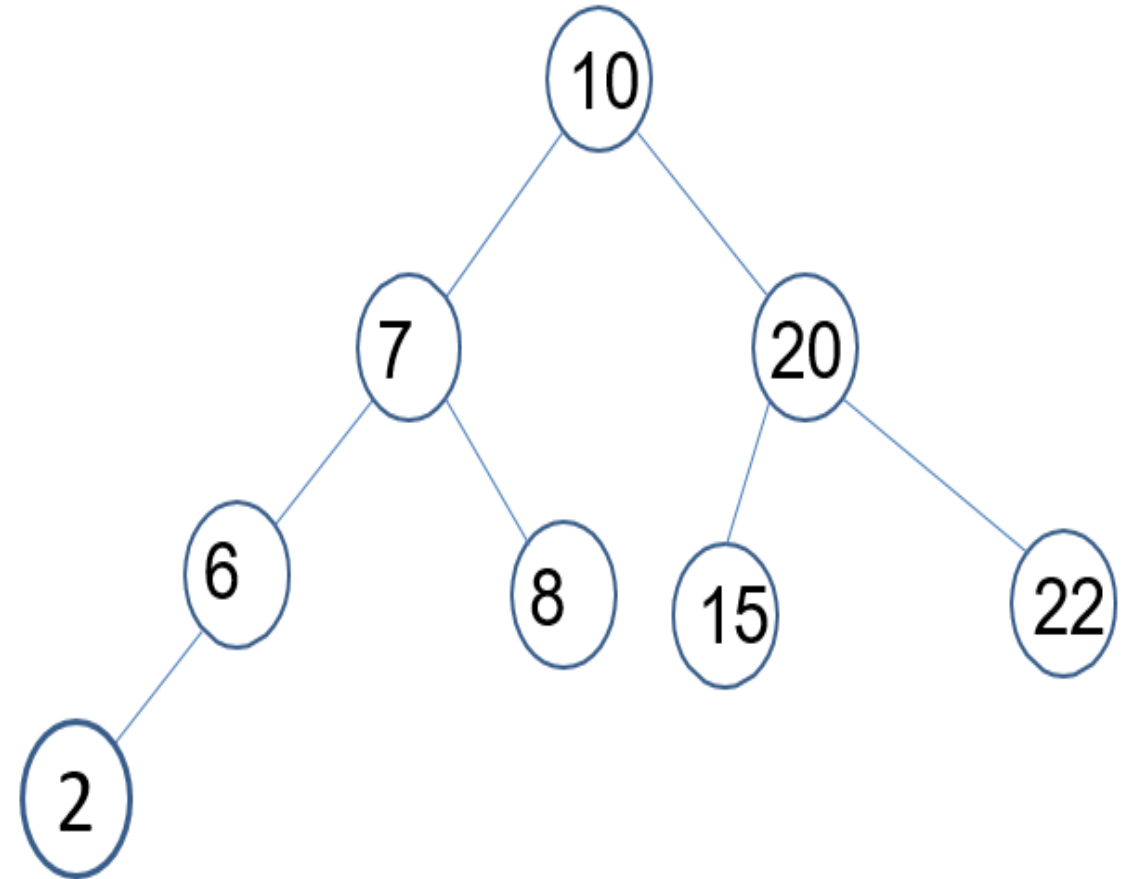
```

// Case 2: Two children
struct Node* temp = findMin(root->right); // Find in-order successor
root->data = temp->data; // Copy successor's data to current node
root->right = deleteNode(root->right, temp->data); // Delete the successor
}
return root;
}

```

3. Min & Max Element

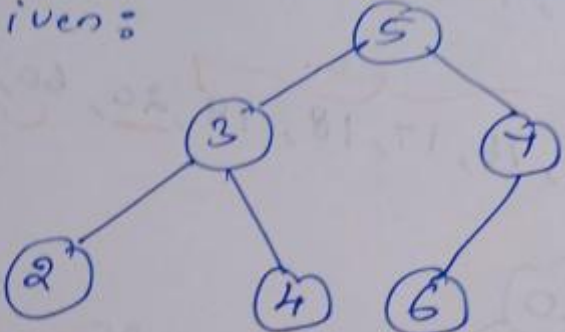
```
struct BstNode {  
    int data;  
    BstNode* left;  
    BstNode* right;  
};  
  
int FindMin(BstNode* root) {  
    if(root == NULL) {  
        cout<<"Error: Tree is empty\n";  
        return -1;  
    }  
    while(root->left != NULL) {  
        root = root->left;  
    }  
    return root->data;  
}
```



4. Kth smallest Element

k^{th} smallest element in BST.

Given:



Method 1

It's inorder: $[2, 3, 4, 5, 6, 7]$
(in ascending order)

Return 5th smallest element: 6

Return 1st smallest element: 2

Return 2nd smallest element: 3

```
void inorderTraversal(struct TreeNode*  
root, int k) {  
    if (root == NULL) {  
        return;  
    }  
    inorderTraversal(root->left, k);  
    count++;  
    if (count == k) {  
        result = root->val;  
        return;  
    }  
    inorderTraversal(root->right, k);  
}
```

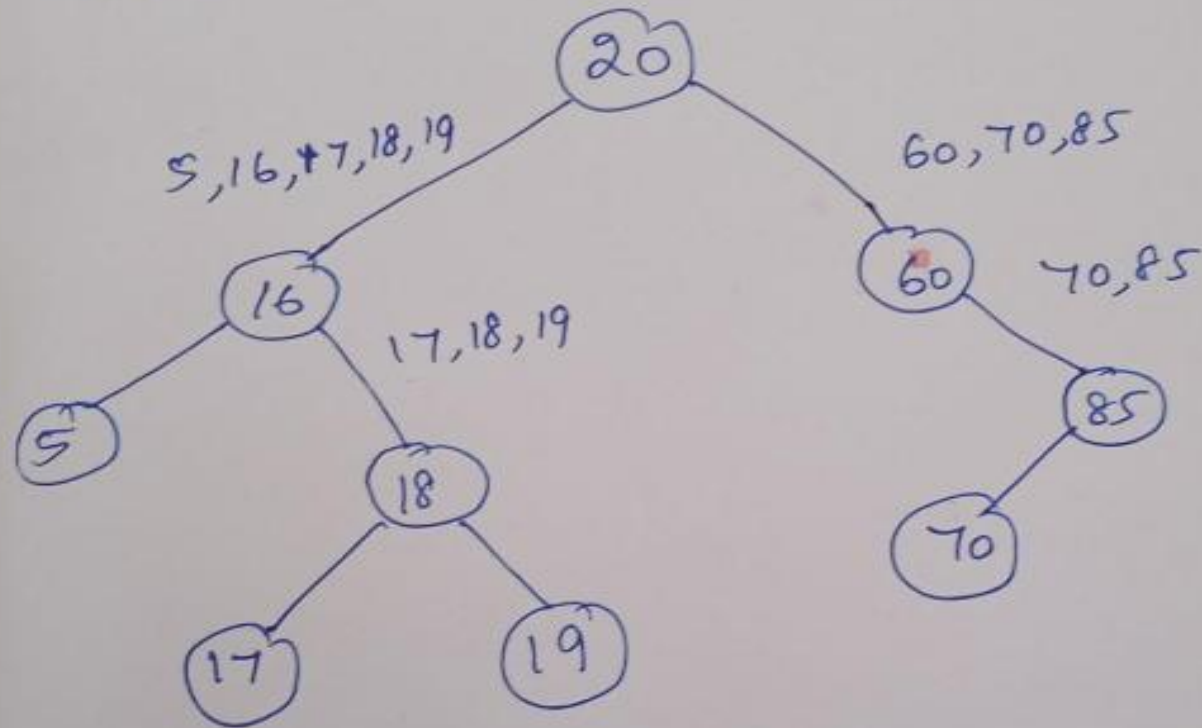
```
int kthSmallest(struct TreeNode*  
root, int k) {  
    count = 0;  
    result = -1;  
    inorderTraversal(root, k);  
    return result;  
}
```

Construct BST from preorder traversal:

preorder: 20, 16, 5, 18, 17, 19, 60, 85, 70

Soln:

Its inorder is: 5, 16, 17, 18, 19, 20, 60, 70, 85



Construct BST from given
post-order

Post order : 5, 17, 19, 18, 16, 70, 85,
60, 20

Soln:

In order : 5, 16, 17, 18, 19, 20, 60, 70, 85

